AprilTag标签识别

AprilTag标签识别
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- find_apriltags()
- 类 `AprilTag`
  - 构造函数
  - `corners`
  - `rect`
  - `x`
  - `y`
  - `w`
  - `h`
  - `id`
  - `family`
  - `cx`
  - `cy`
  - `rotation`
  - `decision_margin`
  - `hamming`
  - `goodness`
  - `x_translation`
  - `y_translation`
  - `z_translation`
  - `x_rotation`
  - `y_rotation`
  - `z_rotation`

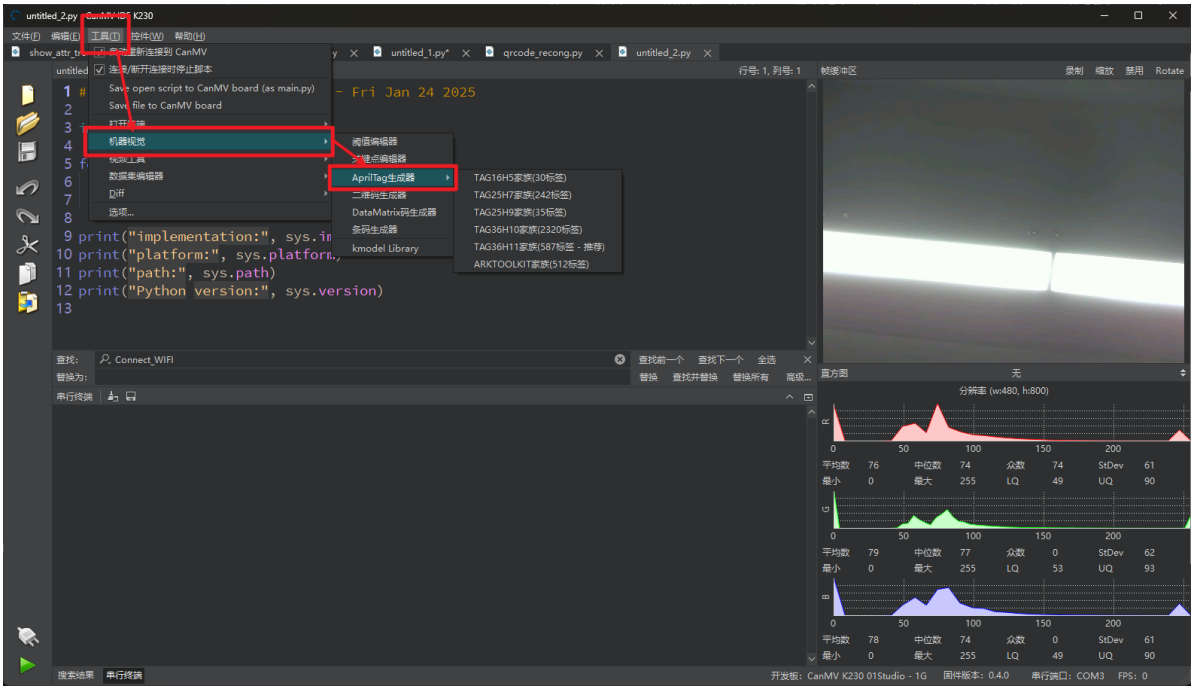
例程实验效果简介

本节我们来识别一下AprilTag标签

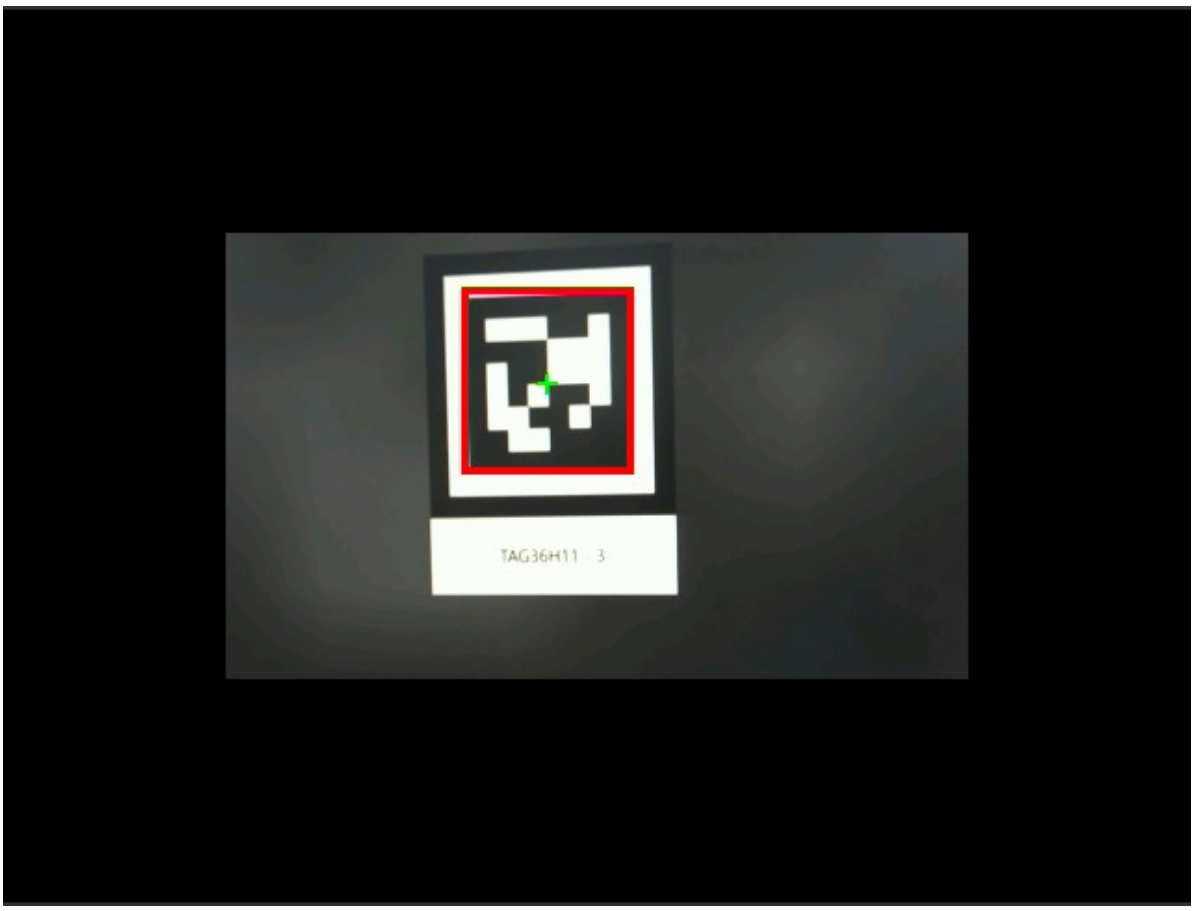
> AprilTag是一种用于视觉识别的二维条形码，广泛应用于机器人导航、增强现实、以及相机标定等领域。它最初由乔治亚理工学院的研究人员开发。AprilTag的设计目的是提供一种可靠且易于识别的标签系统，能够在复杂环境中高效工作。
>
> 以下是AprilTag的一些主要特点：
>
> 1. 高精度识别：AprilTag利用图像处理技术，能够在多种光照条件和背景下准确识别和定位标签。
> 2. 信息编码：每个AprilTag可以编码不同的信息，通常是一个独特的整数ID，允许系统对多个标签进行区分。
> 3. 快速检测：该系统能够快速检测标签位置和朝向，这对于实时应用尤为重要，比如机器人定位和导航。
> 4. 抗干扰性：AprilTag具有较强的抗干扰能力，能够在存在噪声或其他干扰物体的情况下，依然保持高识别率。
> 5. 开源支持：AprilTag的实现是开源的，开发者可以自由使用和修改，从而促进了其在各类项目中的应用。

> 6. 易于制作：用户可以方便地打印出AprilTag，制作标签用于实验或应用。
>
> 由于这些优点，AprilTag被广泛应用于机器人、无人机、AR/VR等多个领域，成为视觉定位和物体识别的热门选择。

我们已经生成了tag36h11家族的587个标签，放在【K230 教程\0.开发板资料\AprilTags】目录下，可以直接用来作识别。大家也可以使用CanMV IDE上自带的AprilTag生成器自行生成其它家族的AprilTag标签



运行例程代码，将摄像头对准待识别的AprilTag码，可以看到AprilTag码被红框和绿色的十字箭头标示出来了

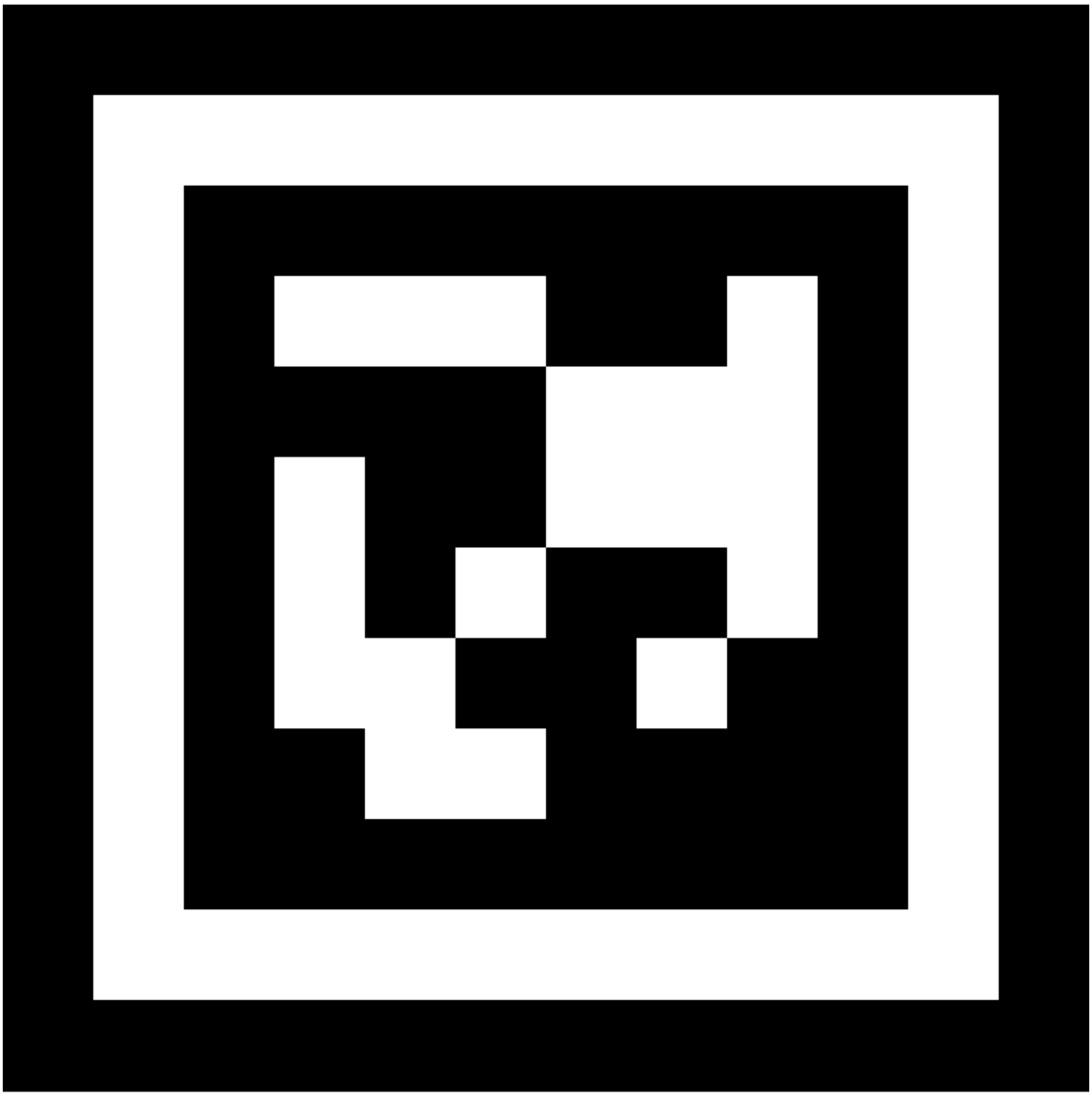


在CanMV IDE的串行终端输出处，可以看到输出了如下信息：

> Tag Family TAG36H11, Tag ID 0, rotation 2.835588 (degrees)

其中rotation代表着旋转偏移

示例图片



TAG36H11 - 3

代码讲解

本节我们要用的的外设主要是摄像头模块

AprilTag的检测和识别 由 K230中的 find_apriltags() 方法实现，该方法属于image模块

完整代码

```
# Import required modules
# 导入所需模块
import time, math, os, gc

from media.sensor import *
from media.display import *
from media.media import *

from libs.YbProtocol import YbProtocol
from ybUtils.YbUart import YbUart
# uart = None
uart = YbUart(baudrate=115200)
pto = YbProtocol()


# AprilTag code supports processing up to 6 tag families simultaneously
# The returned tag object will have its tag family and ID within that family
# AprilTag代码最多支持同时处理6种tag家族
# 返回的tag标记对象包含其所属家族及在该家族中的ID

# Initialize tag families bitmask
# 初始化tag家族位掩码
tag_families = 0
tag_families |= image.TAG16H5    # Comment out to disable this family / 注释掉以禁用此家族
tag_families |= image.TAG25H7    # Comment out to disable this family / 注释掉以禁用此家族
tag_families |= image.TAG25H9    # Comment out to disable this family / 注释掉以禁用此家族
tag_families |= image.TAG36H10   # Comment out to disable this family / 注释掉以禁用此家族
tag_families |= image.TAG36H11   # Comment out to disable this family (default) / 注释掉以禁用此家族(默认)
tag_families |= image.ARTOOLKIT  # Comment out to disable this family / 注释掉以禁用此家族

def family_name(tag):
    """
    Get the family name of a tag
    获取tag的家族名称

    Args:
        tag: AprilTag object / AprilTag对象
    Returns:
        str: Name of the tag family / tag家族名称
    """
    family_dict = {
        image.TAG16H5: "TAG16H5",
        image.TAG25H7: "TAG25H7",
        image.TAG25H9: "TAG25H9",
        image.TAG36H10: "TAG36H10",
        image.TAG36H11: "TAG36H11",
        image.ARTOOLKIT: "ARTOOLKIT"
    }
    return family_dict.get(tag.family())
```

```
# Initialize camera sensor
# 初始化摄像头传感器
sensor = Sensor()
sensor.reset()
sensor.set_framesize(width=400, height=240)
sensor.set_pixformat(Sensor.RGB565)

# Initialize display
# 初始化显示
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
#Display.init(Display.VIRT, sensor.width(), sensor.height())

# Initialize media manager and start sensor
# 初始化媒体管理器并启动传感器
MediaManager.init()
sensor.run()

# Create clock for FPS calculation
# 创建时钟用于FPS计算
clock = time.clock()

# Main loop
# 主循环
while True:
    clock.tick()

    # Capture image
    # 捕获图像
    img = sensor.snapshot()

    # Find and process AprilTags
    # 查找和处理AprilTags
    for tag in img.find_apriltags(families=tag_families):
        # Draw rectangle and cross on detected tag
        # 在检测到的tag上绘制矩形和十字
        img.draw_rectangle(tag.rect(), color=(255, 0, 0), thickness=4)
        img.draw_cross(tag.cx(), tag.cy(), color=(0, 255, 0), thickness=2)

        # Print tag information
        # 打印tag信息
        print_args = (family_name(tag), tag.id(), (180 * tag.rotation()) /
math.pi)
        print("Tag Family %s, Tag ID %d, rotation %f (degrees)" % print_args)

        x, y, w, h = tag.rect()
        pto_data = pto.get_apriltag_data(x, y, w, h, tag.id(),
(180*tag.rotation())/math.pi)
        uart.send(pto_data)
        print(pto_data)
        break

    # Display image centered on screen
    # 在屏幕中央显示图像
    Display.show_image(img, x=120, y=120)

    # Print frames per second
    # 打印帧率
```

```
    print(clock.fps())
```

代码结构

本节例程主要功能和结构如下：

基本设置部分：

- 首先导入必要的模块，包括时间、数学计算、操作系统和垃圾回收等功能
- 初始化了6种不同的AprilTag标记家族，可以通过注释来启用或禁用特定家族
- 定义了一个family_name函数，用于将tag家族的数字ID转换为可读的字符串名称

硬件初始化：

- 设置摄像头传感器参数：分辨率为1280x960，实际工作分辨率设为320x240，使用RGB565色彩格式
- 初始化显示器：使用ST7701显示器，分辨率640x480
- 初始化媒体管理器并启动摄像头传感器

主循环功能：

- 持续捕获图像帧
- 在每一帧中查找AprilTag标记
- 对于找到的每个标记：
    - 用红色矩形框标记tag位置
    - 用绿色十字标记tag中心点
    - 打印出tag的家族名称、ID号和旋转角度
- 将处理后的图像显示在屏幕中央
- 显示当前的帧率（FPS）

find_apriltags()

```
image.find_apriltags([roi[, families=image.TAG36H11[, fx[, fy[, cx[, cy]]]]]])
```

该函数查找指定 ROI 内的所有 AprilTag，并返回一个包含 `image.apriltag` 对象的列表。有关更多信息，请参考 `image.apriltag` 对象的相关文档。

与二维码相比，AprilTags 可以在更远的距离、较差的光照条件和更扭曲的图像环境下被有效检测。AprilTags 能够应对各种图像失真问题，而二维码则不能。因此，AprilTags 仅将数字 ID 编码作为其有效载荷。

此外，AprilTags 还可用于定位。每个 `image.apriltag` 对象将返回其三维位置信息和旋转角度。位置信息由 `fx`、`fy`、`cx` 和 `cy` 决定，分别表示图像在 X 和 Y 方向上的焦距和中心点。

> 可以使用 OpenMV IDE 内置的标签生成器工具创建 AprilTags。该工具可生成可打印的 8.5"x11" 格式的 AprilTags。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。
- `families` 是要解码的标签家族的位掩码，以逻辑或形式表示：
    - `image.TAG16H5`
    - `image.TAG25H7`

- `image.TAG25H9`
- `image.TAG36H10`
- `image.TAG36H11`
- `image.ARTOOLKIT`

默认设置为最常用的 `image.TAG36H11` 标签家族。请注意，启用每个标签家族都会稍微降低 `find_apriltags` 的速度。

- `fx` 是以像素为单位的相机 X 方向的焦距。标准 OpenMV Cam 的值为 ((2.8 / 3.984) \times 656)，该值通过毫米计的焦距值除以 X 方向上感光元件的长度，再乘以 X 方向上感光元件的像素数量（针对 OV7725 感光元件）。
- `fy` 是以像素为单位的相机 Y 方向的焦距。标准 OpenMV Cam 的值为 ((2.8 / 2.952) \times 488)，该值通过毫米计的焦距值除以 Y 方向上感光元件的长度，再乘以 Y 方向上感光元件的像素数量（针对 OV7725 感光元件）。
- `cx` 是图像的中心，即 `image.width()/2`，而非 `roi.w()/2`。
- `cy` 是图像的中心，即 `image.height()/2`，而非 `roi.h()/2`。

注意： 不支持压缩图像和 Bayer 图像。

类 `AprilTag`

AprilTag 对象由 `image.find_apriltags` 函数返回。

构造函数

```
class image.apriltag
```

请使用 `image.find_apriltags()` 函数创建此对象。

`corners`

```
apriltag.corners()
```

该方法返回一个包含 AprilTag 四个角的元组列表，每个元组格式为 (x, y)。四个角的顺序通常为从左上角开始，按顺时针方向排列。

`rect`

```
apriltag.rect()
```

该方法返回一个矩形元组 (x, y, w, h)，可用于其他图像处理方法，如 `image.draw_rectangle` 中的 AprilTag 边界框。

`x`

```
apriltag.x()
```

该方法返回 AprilTag 边界框的 x 坐标 (int)。您也可以通过索引 [0] 获取该值。

`y`

```
apriltag.y()
```

该方法返回 AprilTag 边界框的 y 坐标 (int)。您也可以通过索引 [1] 获取该值。

`w`

```
apriltag.w()
```

该方法返回 AprilTag 边界框的宽度 (int)。您也可以通过索引 [2] 获取该值。

`h`

```
apriltag.h()
```

该方法返回 AprilTag 边界框的高度 (int)。您也可以通过索引 [3] 获取该值。

`id`

```
apriltag.id()
```

该方法返回 AprilTag 的数字 ID。

- TAG16H5 -> 0 到 29
- TAG25H7 -> 0 到 241
- TAG25H9 -> 0 到 34
- TAG36H10 -> 0 到 2319
- TAG36H11 -> 0 到 586
- ARTOOLKIT -> 0 到 511

您也可以通过索引 [4] 获取该值。

`family`

```
apriltag.family()
```

该方法返回 AprilTag 的数字家族。

- image.TAG16H5
- image.TAG25H7
- image.TAG25H9
- image.TAG36H10
- image.TAG36H11
- image.ARTOOLKIT

您也可以通过索引 [5] 获取该值。

cx

```
apriltag.cx()
```

该方法返回 AprilTag 的中心 x 坐标 (int)。您也可以通过索引 [6] 获取该值。

cy

```
apriltag.cy()
```

该方法返回 AprilTag 的中心 y 坐标 (int)。您也可以通过索引 [7] 获取该值。

rotation

```
apriltag.rotation()
```

该方法返回 AprilTag 的旋转角度，以弧度计量 (int)。您也可以通过索引 [8] 获取该值。

decision_margin

```
aprilt

ag.decision_margin()
```

该方法返回 AprilTag 的决策边际，反映了对检测的置信度 (float)。您也可以通过索引 [9] 获取该值。

hamming

```
apriltag.hamming()
```

该方法返回 AprilTag 可接受的最大汉明距离（即可接受的数位误差）。具体如下：

- TAG16H5：最多可接受 0 位错误
- TAG25H7：最多可接受 1 位错误
- TAG25H9：最多可接受 3 位错误
- TAG36H10：最多可接受 3 位错误
- TAG36H11：最多可接受 4 位错误
- ARTOOLKIT：最多可接受 0 位错误

您可以通过索引 [10] 获取该值。

goodness

```
apriltag.goodness()
```

该方法返回 AprilTag 图像的色饱和度，取值范围为 0.0 到 1.0，其中 1.0 表示最佳状态。

> 目前该数值通常为 0.0。未来我们计划启用名为“标签细化”的功能，以支持对更小的 AprilTag 的检测。然而，目前此功能可能会导致帧速率降低至 1 FPS 以下。

您可以通过索引 [11] 获取该值。

x_translation

```
apriltag.x_translation()
```

该方法返回摄像机在 x 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 x 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从左至右。

您可以通过索引 [12] 获取该值。

y_translation

```
apriltag.y_translation()
```

该方法返回摄像机在 y 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 y 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从上至下。

您可以通过索引 [13] 获取该值。

z_translation

```
apriltag.z_translation()
```

该方法返回摄像机在 z 方向上的位移，单位未知。

此方法对于确定远离摄像机的 AprilTag 的位置非常有用。然而，AprilTag 的大小以及您使用的镜头等因素都会影响 z 单位的归属。为了方便使用，我们建议您使用查找表将该方法的输出转换为适用于您应用程序的信息。

注意：此处的方向为从前至后。

您可以通过索引 [14] 获取该值。

x_rotation

```
apriltag.x_rotation()
```

该方法返回 AprilTag 在 x 平面上的旋转角度，以弧度计量。例如，若从左至右移动摄像头观察 AprilTag，则可应用此方法。

您可以通过索引 [15] 获取该值。

y_rotation

```
apriltag.y_rotation()
```

该方法返回 AprilTag 在 y 平面上的旋转角度，以弧度计量。例如，若从上至下移动摄像头观察 AprilTag，则可应用此方法。

您可以通过索引 [16] 获取该值。

z_rotation

```
apriltag.z_rotation()
```

该方法返回 AprilTag 在 z 平面上的旋转角度，以弧度计量。例如，若旋转摄像头观察 AprilTag，则可应用此方法。

注意：此方法为 `apriltag.rotation()` 的重命名版本。

您可以通过索引 [17] 获取该值。